École Supérieure Privée d'Ingénieurs de Monastir

Année Universitaire 2025 – 2026

TumorSight

Système de Détection et de Classification des Tumeurs Cérébrales Basé sur l'Intelligence Artificielle

Rapport de Projet

Réalisé par :
ZAGHDOUD Loujein

Encadré par :
HAFFAR Nafaa

Spécialité :
Intelligence Artificielle / Data Science

2026

Résumé

L'intelligence artificielle est devenue l'une des technologies les plus importantes dans les systèmes de santé modernes. Ces dernières années, les techniques de Deep Learning ont montré des performances remarquables dans l'analyse des images médicales, notamment dans la détection et la classification des maladies à partir des images IRM.

L'objectif de ce projet est de développer une plateforme intelligente appelée **TumorSight**, capable de détecter automatiquement les tumeurs cérébrales et de classifier leurs stades à l'aide des techniques d'intelligence artificielle. Le système combine des modèles de Deep Learning avec un tableau de bord interactif afin de simplifier l'analyse des images médicales et d'assister les professionnels de santé lors du diagnostic.

TumorSight suit un pipeline de prédiction en deux étapes. Premièrement, un modèle de classification binaire détermine si une tumeur est présente dans l'image IRM téléchargée. Si une tumeur est détectée, un second modèle de classification multi-classes prédit le stade de la tumeur. La plateforme fournit également des scores de confiance, des visualisations graphiques et une exportation des résultats.

Le projet a été développé avec Python, TensorFlow, Keras, Streamlit, NumPy, Plotly, Docker et GitHub. TumorSight démontre comment l'intelligence artificielle peut contribuer à l'amélioration de l'assistance médicale en réduisant le temps d'analyse et en automatisant l'interprétation des images IRM.

Mots-clés : Intelligence Artificielle, Deep Learning, CNN, EfficientNet, IRM, Imagerie Médicale, Détection de Tumeurs.

Table des matières

Résumé **1**

Introduction Générale **4**

1 Contexte du Projet et Problématique **5**

1.1 Objectifs du Projet 5

2 Étude Bibliographique **6**

3 Architecture du Système **7**

3.1 Couches Principales du Système 7

3.2 Workflow du Système 7

4 Dataset et Prétraitement des Images **9**

4.1 Description du Dataset 9

4.2 Prétraitement des Images 9

4.2.1 Conversion RGB 9

4.2.2 Redimensionnement 9

4.2.3 Normalisation 10

4.2.4 Augmentation des Données 10

5 Modèles de Deep Learning **11**

5.1 Principe Général des Réseaux CNN 11

5.1.1 Couche de Convolution 11

5.1.2 Fonction d'Activation ReLU 11

5.1.3 Couche MaxPooling 12

5.1.4 Couche Flatten 12

5.1.5 Couches Dense 12

5.1.6 Couche Softmax 12

5.2 Modèle CNN Baseline 12

5.2.1 Objectif du CNN Baseline 12

5.2.2 Architecture du CNN 12
5.2.3 Configuration d'Entraînement 13
5.2.4 Avantages du CNN Baseline 13
5.2.5 Limites du CNN Baseline 13
5.3 Modèle EfficientNet 14
5.3.1 Principe d'EfficientNet 14
5.3.2 Transfer Learning avec EfficientNet 14
5.3.3 Étapes du Transfer Learning 14
5.3.4 Architecture EfficientNet Adaptée 15
5.3.5 Pourquoi Utiliser EfficientNet ? 15
5.3.6 Comparaison CNN Baseline et EfficientNet 15
5.4 Pipeline de Prédiction 16
5.5 Scores de Confiance 16

6 Implémentation du Tableau de Bord 17

7 Résultats et Évaluation 19
7.1 Accuracy 19
7.2 Precision 19
7.3 Recall 19
7.4 F1-Score 19
7.5 ROC-AUC 19
7.6 Analyse des Résultats 20

8 Déploiement et Gestion des Versions 21
8.1 Avantages du Déploiement avec Docker 21

Conclusion Générale 22

Références 23

Introduction Générale

L'évolution rapide de l'intelligence artificielle a transformé plusieurs domaines, notamment la santé, la finance, le transport et l'éducation. Dans le domaine médical, l'analyse automatique des images médicales représente aujourd'hui l'une des applications les plus prometteuses du Deep Learning.

Les images IRM sont largement utilisées pour détecter les tumeurs cérébrales et suivre l'évolution des maladies. Cependant, leur interprétation manuelle reste une tâche complexe qui dépend fortement de l'expérience du radiologue. Une erreur ou un retard dans le diagnostic peut avoir des conséquences importantes sur la prise en charge du patient.

Dans ce contexte, les réseaux de neurones convolutionnels ont montré une grande efficacité dans la classification des images médicales. Ils permettent d'extraire automatiquement les caractéristiques importantes d'une image, comme les contours, les textures et les formes anormales.

Le projet TumorSight vise à développer une plateforme intelligente capable de détecter automatiquement la présence d'une tumeur cérébrale et de classifier son stade à partir d'images IRM. Le système propose une interface simple, interactive et accessible permettant d'afficher les résultats de prédiction, les scores de confiance et les probabilités associées.

Ce rapport présente les différentes étapes du projet : contexte, problématique, étude bibliographique, architecture du système, dataset, prétraitement, modèles de Deep Learning, implémentation du tableau de bord, résultats, évaluation, déploiement et perspectives d'amélioration.

1 Contexte du Projet et Problématique

L'imagerie médicale joue un rôle fondamental dans les systèmes de santé modernes. Les images IRM permettent aux professionnels de santé de visualiser les tissus internes du cerveau et de détecter différentes anomalies telles que les tumeurs, les infections et les troubles neurologiques.

Les hôpitaux génèrent quotidiennement un grand nombre d'images médicales. Par conséquent, les radiologues peuvent être confrontés à une charge de travail importante, ce qui peut augmenter le temps de diagnostic et provoquer des erreurs d'interprétation.

Dans le cas des tumeurs cérébrales, la détection précoce est très importante, car elle permet d'améliorer les chances de traitement et de réduire les risques de complications. Un faux négatif, c'est-à-dire une tumeur non détectée, peut représenter un risque important pour le patient.

La problématique principale de TumorSight consiste donc à développer une plateforme intelligente capable de détecter automatiquement les tumeurs cérébrales et de classifier leurs stades à partir d'images IRM.

1.1 Objectifs du Projet

Les objectifs principaux de ce projet sont :

— Détecter automatiquement la présence ou l'absence d'une tumeur cérébrale.
— Classifier le stade de la tumeur détectée.
— Fournir un score de confiance pour chaque prédiction.
— Simplifier l'analyse des images IRM à travers une interface interactive.
— Réduire le temps nécessaire à l'interprétation des images médicales.
— Exporter les résultats au format CSV pour une analyse ultérieure.

2 Étude Bibliographique

L'intelligence artificielle occupe une place de plus en plus importante dans le domaine médical grâce à sa capacité à traiter de grandes quantités de données. Elle est utilisée dans plusieurs applications telles que la prédiction des maladies, l'aide au diagnostic, la classification des images et la segmentation médicale.

Le Deep Learning est une branche de l'intelligence artificielle basée sur des réseaux de neurones profonds. Contrairement aux méthodes classiques de Machine Learning, les modèles de Deep Learning peuvent apprendre automatiquement les caractéristiques importantes à partir des données.

Les réseaux de neurones convolutionnels, appelés CNN, sont particulièrement adaptés au traitement des images. Ils sont largement utilisés dans la détection des tumeurs, la classification des cancers, la segmentation des organes et l'aide au diagnostic médical.

Dans ce projet, TumorSight utilise des modèles CNN et EfficientNet afin d'analyser les images IRM et produire des prédictions fiables.

3 Architecture du Système

TumorSight suit une architecture modulaire composée de plusieurs composants interconnectés. Cette organisation permet de séparer les différentes responsabilités du système et de faciliter la maintenance du projet.

3.1 Couches Principales du Système

L'architecture comprend les couches suivantes :

— **Couche Interface Utilisateur :** elle permet à l'utilisateur de télécharger une image IRM et de visualiser les résultats.

— **Couche Prétraitement :** elle prépare l'image avant son passage dans le modèle.

— **Couche Prédiction IA :** elle contient les modèles de Deep Learning.

— **Couche Visualisation :** elle affiche les probabilités et les scores de confiance.

— **Couche Exportation :** elle permet de sauvegarder les résultats au format CSV.

3.2 Workflow du Système

Le fonctionnement du système commence lorsque l'utilisateur télécharge une image IRM via le tableau de bord Streamlit. L'image est ensuite convertie, redimensionnée et normalisée. Après le prétraitement, elle est envoyée vers le modèle de détection binaire afin de déterminer si une tumeur est présente ou non.

Si aucune tumeur n'est détectée, le système affiche le résultat final. Si une tumeur est détectée, l'image est transmise au second modèle chargé de classifier le stade de la tumeur. Le tableau de bord affiche ensuite les résultats de prédiction, les scores de confiance et les graphiques des probabilités.

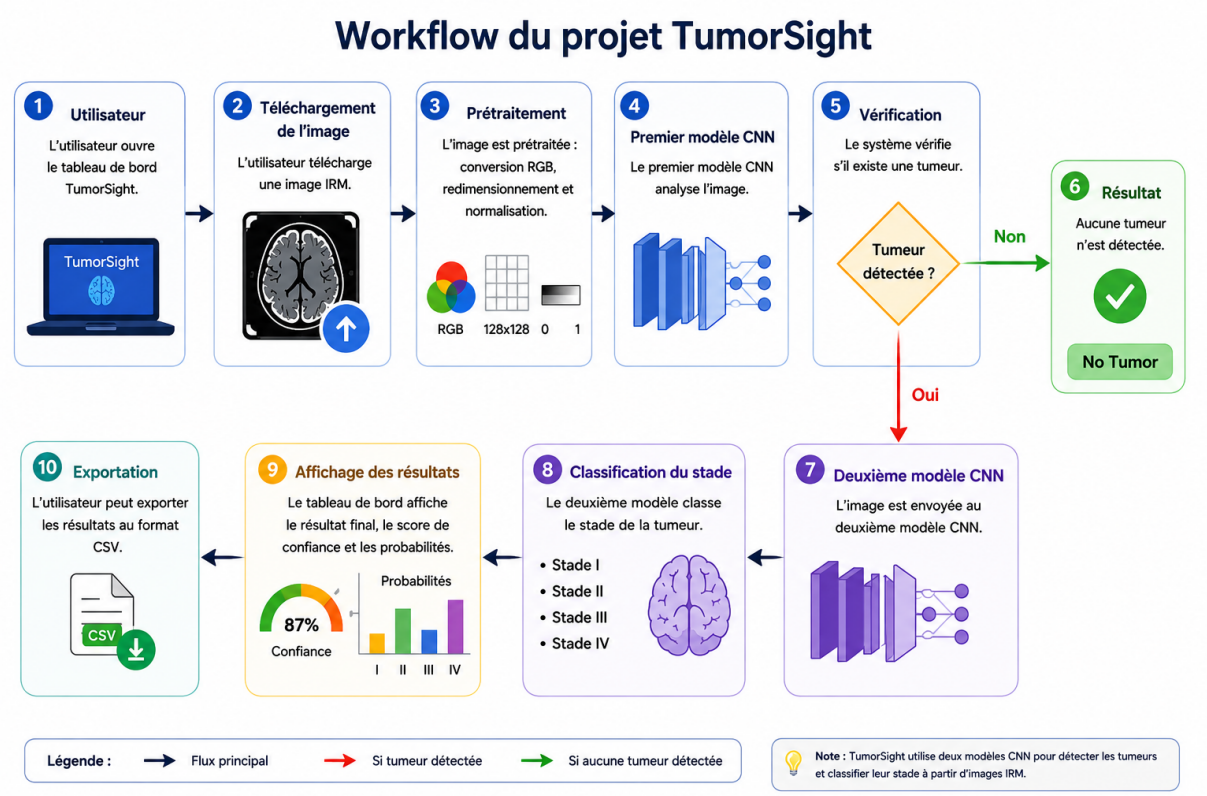

FIGURE 3.1 – Workflow du projet TumorSight

4 Dataset et Prétraitement des Images

La qualité d'un système de Deep Learning dépend fortement de la qualité des données utilisées. Dans ce projet, TumorSight utilise des images IRM représentant des cas sains ainsi que des cas contenant des tumeurs.

4.1 Description du Dataset

Le dataset contient des images médicales de type IRM. Ces images sont utilisées pour entraîner les modèles à reconnaître la présence d'une tumeur et à distinguer les différentes classes tumorales.

Les données sont organisées en plusieurs ensembles :

- **Ensemble d'entraînement :** utilisé pour apprendre les paramètres du modèle.
- **Ensemble de validation :** utilisé pour suivre les performances pendant l'entraînement.
- **Ensemble de test :** utilisé pour évaluer le modèle final.

4.2 Prétraitement des Images

Avant d'être analysées par les modèles, les images passent par plusieurs étapes de prétraitement.

4.2.1 Conversion RGB

Les images sont converties au format RGB afin de garantir leur compatibilité avec les modèles CNN et EfficientNet.

4.2.2 Redimensionnement

Toutes les images sont redimensionnées à une taille fixe. Pour le modèle CNN classique, les images sont redimensionnées à 128×128 pixels. Pour EfficientNet, les images peuvent être redimensionnées à 224×224 pixels selon l'architecture utilisée.

4.2.3 Normalisation

Les valeurs des pixels sont normalisées entre 0 et 1. Cette étape améliore la stabilité de l'entraînement et accélère la convergence du modèle.

4.2.4 Augmentation des Données

L'augmentation des données est utilisée pour améliorer la généralisation du modèle. Elle consiste à appliquer des transformations légères aux images d'entraînement.

— Rotation légère des images.
— Zoom.
— Flip horizontal.
— Modification légère du contraste.

Ces transformations permettent de simuler différentes conditions d'acquisition des images IRM et de réduire le risque de surapprentissage.

```
img = Image.open(uploaded_file).convert("RGB")
img = img.resize((128, 128))
img = np.array(img, dtype=np.float32) / 255.0
```

5 Modèles de Deep Learning

Cette partie représente le cœur du projet TumorSight. Les modèles de Deep Learning permettent d'analyser automatiquement les images IRM et de produire une prédiction sur la présence d'une tumeur ou sur son stade.

Dans ce projet, deux types de modèles ont été étudiés :

— Un modèle CNN classique développé à partir de zéro.

— Un modèle EfficientNet basé sur le Transfer Learning.

5.1 Principe Général des Réseaux CNN

Un réseau de neurones convolutionnel, appelé CNN, est une architecture spécialement conçue pour le traitement des images. Son objectif est d'extraire automatiquement les caractéristiques importantes d'une image sans avoir besoin de les définir manuellement.

Dans le cas des images IRM, un CNN peut apprendre à reconnaître des formes, des contours, des zones sombres ou claires, des textures et des anomalies liées à la présence d'une tumeur.

Un CNN est généralement composé de plusieurs types de couches :

5.1.1 Couche de Convolution

La couche de convolution applique des filtres sur l'image afin de détecter des motifs locaux. Ces filtres peuvent détecter les contours, les formes, les textures et les zones suspectes. Chaque filtre produit une carte de caractéristiques appelée feature map.

5.1.2 Fonction d'Activation ReLU

La fonction ReLU introduit de la non-linéarité dans le modèle. Elle permet au réseau d'apprendre des relations complexes entre les pixels de l'image.

5.1.3 Couche MaxPooling

La couche MaxPooling réduit la taille des cartes de caractéristiques tout en conservant les informations les plus importantes. Cette étape permet de réduire le coût de calcul et de rendre le modèle plus robuste aux petites variations dans les images.

5.1.4 Couche Flatten

La couche Flatten transforme les cartes de caractéristiques en un vecteur unique. Ce vecteur est ensuite transmis aux couches denses pour réaliser la classification finale.

5.1.5 Couches Dense

Les couches Dense combinent les informations extraites par les couches précédentes afin de prendre une décision. Elles jouent un rôle important dans l'interprétation des caractéristiques extraites.

5.1.6 Couche Softmax

La couche Softmax est utilisée dans la classification multi-classes. Elle transforme les sorties du modèle en probabilités. La classe ayant la probabilité la plus élevée devient la prédiction finale.

5.2 Modèle CNN Baseline

Le premier modèle utilisé dans TumorSight est un CNN classique construit à partir de zéro. Ce modèle sert de référence de départ afin d'évaluer les performances d'une architecture simple avant d'utiliser des modèles plus avancés.

5.2.1 Objectif du CNN Baseline

L'objectif de ce modèle est de détecter les caractéristiques visuelles importantes dans les images IRM et de classifier les images selon les catégories définies. Il permet de vérifier si un réseau simple peut apprendre efficacement à distinguer les images contenant une tumeur des images normales.

5.2.2 Architecture du CNN

Couche	**Configuration**	**Rôle**

Input	Image RGB 128×128×3	Entrée du modèle
Conv2D	32 filtres, taille 3×3, ReLU	Extraction des motifs simples
MaxPooling2D	Taille 2×2	Réduction spatiale
Conv2D	64 filtres, taille 3×3, ReLU	Extraction des caractéristiques plus complexes
MaxPooling2D	Taille 2×2	Réduction du nombre de paramètres
Flatten	Transformation en vecteur	Préparation à la classification
Dense	128 neurones, ReLU	Interprétation des caractéristiques
Dropout	30%	Réduction du surapprentissage
Dense	Softmax	Classification finale

5.2.3 Configuration d'Entraînement

Le CNN Baseline a été entraîné avec l'optimiseur Adam. La fonction de perte utilisée est la Categorical Crossentropy, adaptée aux problèmes de classification multi-classes.

— **Optimiseur :** Adam.
— **Fonction de perte :** Categorical Crossentropy.
— **Batch size :** 32.
— **Nombre d'époques :** selon l'entraînement.
— **Métrique principale :** Accuracy.

5.2.4 Avantages du CNN Baseline

Le CNN Baseline présente plusieurs avantages :

— Architecture simple et facile à comprendre.
— Temps d'entraînement raisonnable.
— Bonne base de comparaison avec les modèles avancés.
— Extraction automatique des caractéristiques.

5.2.5 Limites du CNN Baseline

Malgré sa simplicité, le CNN Baseline peut présenter certaines limites. Comme il est entraîné à partir de zéro, il a besoin d'un grand nombre d'images pour généraliser

correctement. Il peut aussi être sensible au surapprentissage si le dataset est limité.

5.3 Modèle EfficientNet

EfficientNet est une famille de réseaux de neurones convolutionnels modernes conçus pour obtenir de bonnes performances tout en utilisant moins de paramètres que d'autres architectures profondes.

5.3.1 Principe d'EfficientNet

EfficientNet repose sur une stratégie appelée compound scaling. Cette stratégie permet d'augmenter simultanément trois dimensions du réseau :

— La profondeur du réseau.

— La largeur du réseau.

— La résolution des images.

Contrairement aux architectures classiques qui augmentent uniquement le nombre de couches ou la taille des images, EfficientNet équilibre ces trois aspects afin d'obtenir un meilleur rapport entre précision et complexité.

5.3.2 Transfer Learning avec EfficientNet

Dans ce projet, EfficientNet peut être utilisé avec le Transfer Learning. Le Transfer Learning consiste à utiliser un modèle déjà pré-entraîné sur un grand dataset, comme ImageNet, puis à l'adapter au problème spécifique de classification des tumeurs.

Cette approche présente plusieurs avantages :

— Réduction du temps d'entraînement.

— Meilleure performance avec un dataset limité.

— Réutilisation de caractéristiques visuelles déjà apprises.

— Amélioration de la généralisation du modèle.

5.3.3 Étapes du Transfer Learning

Le Transfer Learning avec EfficientNet suit généralement deux étapes.

Feature Extraction

Dans cette première étape, les couches de base d'EfficientNet sont gelées. Cela signifie que leurs poids ne sont pas modifiés pendant l'entraînement. Seules les nouvelles couches ajoutées à la fin du modèle sont entraînées.

L'objectif est d'utiliser EfficientNet comme extracteur de caractéristiques.

Fine-Tuning

Dans la deuxième étape, certaines couches profondes d'EfficientNet sont dégelées. Le modèle est alors réentraîné avec un learning rate faible afin d'adapter progressivement ses poids aux images IRM.

Cette étape permet d'améliorer les performances, surtout lorsque les images médicales sont très différentes des images naturelles utilisées dans ImageNet.

5.3.4 Architecture EfficientNet Adaptée

Composant	**Description**	**Rôle**
Entrée	Image 224×224×3	Format compatible avec EfficientNet
EfficientNet pré-entraîné	Couches convolutionnelles	Extraction des caractéristiques visuelles
GlobalAveragePooling2D	Réduction des dimensions	Transformation des feature maps
Dropout	Régularisation	Réduction du surapprentissage
Dense	Couche fully connected	Classification
Softmax	Probabilités finales	Prédiction de la classe

5.3.5 Pourquoi Utiliser EfficientNet ?

EfficientNet est intéressant dans le cadre de TumorSight car il offre un bon équilibre entre performance et efficacité. Il permet de construire un modèle plus puissant que le CNN Baseline tout en évitant une complexité excessive.

Il est particulièrement utile lorsque le dataset médical n'est pas très volumineux, car le pré-entraînement permet au modèle de bénéficier de connaissances déjà apprises.

5.3.6 Comparaison CNN Baseline et EfficientNet

Critère	**CNN Baseline**	**EfficientNet**
Type	Modèle entraîné from scratch	Modèle pré-entraîné

Taille d'entrée	128×128	224×224
Temps d'entraînement	Moyen	Plus rapide avec Transfer Learning
Besoin en données	Élevé	Moins élevé
Performance	Bonne comme référence	Généralement meilleure
Risque de surapprentissage	Plus élevé	Réduit grâce au pré-entraînement
Complexité	Simple	Plus avancée

5.4 Pipeline de Prédiction

Le pipeline de prédiction de TumorSight suit les étapes suivantes :

1. Téléchargement de l'image IRM.
2. Conversion de l'image au format RGB.
3. Redimensionnement de l'image.
4. Normalisation des pixels.
5. Passage dans le modèle de détection.
6. Calcul des probabilités.
7. Affichage du résultat final.
8. Exportation possible des résultats.

5.5 Scores de Confiance

Le score de confiance permet d'évaluer la fiabilité de la prédiction. Il correspond à la probabilité maximale produite par le modèle. Plus ce score est élevé, plus le modèle est confiant dans sa décision.

Cependant, un score de confiance élevé ne remplace pas l'avis d'un médecin. TumorSight doit être considéré comme un outil d'aide à la décision et non comme un système de diagnostic autonome.

6 Implémentation du Tableau de Bord

L'interface graphique de TumorSight a été développée avec Streamlit. Streamlit est un framework Python qui permet de créer rapidement des applications web interactives pour les projets de Machine Learning et de Deep Learning.

Le tableau de bord a été conçu afin d'offrir un environnement moderne, clair et facile à utiliser pour l'analyse des images médicales.

La plateforme permet à l'utilisateur de télécharger une image IRM, de lancer automatiquement la prédiction et de visualiser les résultats. Le système affiche la probabilité de présence d'une tumeur, le stade prédit, les scores de confiance et les graphiques de probabilités.

TumorSight prend également en charge l'évaluation par lot. Cette fonctionnalité permet d'analyser plusieurs images simultanément, ce qui peut être utile dans les hôpitaux ou les centres médicaux traitant un grand volume d'images.

Le tableau de bord permet aussi l'exportation des résultats au format CSV afin de sauvegarder les prédictions pour une analyse ultérieure.

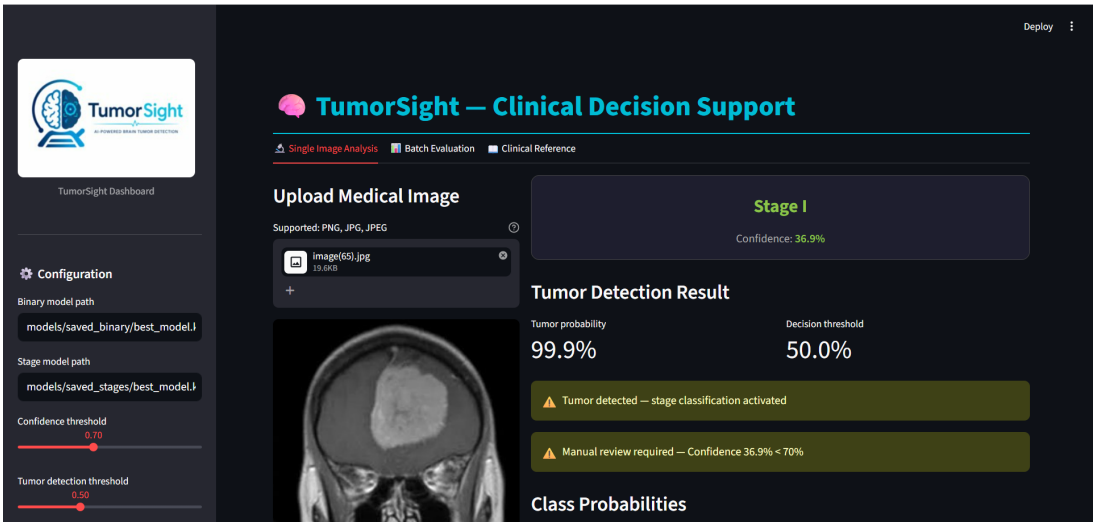

FIGURE 6.1 – Interface d'analyse TumorSight 1.1

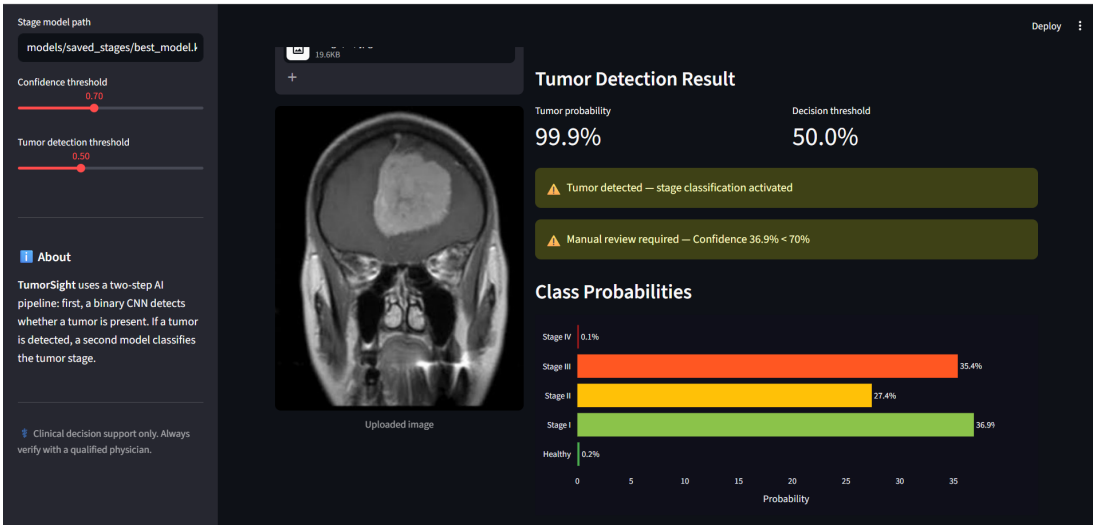

FIGURE 6.2 – Interface d'analyse TumorSight 1.2

7 Résultats et Évaluation

Les performances des modèles de Deep Learning ont été évaluées à l'aide de métriques standards telles que l'Accuracy, la Precision, le Recall, le F1-Score et le ROC-AUC.

7.1 Accuracy

L'Accuracy mesure le pourcentage d'images correctement classifiées par le modèle. Elle donne une idée générale sur la performance globale.

7.2 Precision

La Precision mesure la fiabilité des prédictions positives. Elle indique parmi les images prédites comme positives combien sont réellement positives.

7.3 Recall

Le Recall mesure la capacité du modèle à détecter les vrais cas positifs. Dans un contexte médical, cette métrique est très importante car elle permet de réduire les faux négatifs.

7.4 F1-Score

Le F1-Score combine la Precision et le Recall dans une seule métrique. Il est utile lorsque l'on souhaite équilibrer la fiabilité et la capacité de détection.

7.5 ROC-AUC

Le ROC-AUC mesure la capacité du modèle à distinguer correctement les différentes classes. Une valeur proche de 1 indique une très bonne capacité de discrimination.

7.6 Analyse des Résultats

Les modèles ont montré de bonnes performances dans les tâches de détection des tumeurs et de classification des stades. Le tableau de bord a traité les images IRM et généré des prédictions en temps réel avec succès.

Cependant, certaines limites existent encore. Les performances des modèles dépendent fortement de la qualité et de la quantité des données utilisées. Une validation clinique sur des datasets plus larges reste nécessaire avant toute utilisation réelle dans un environnement médical.

8 Déploiement et Gestion des Versions

Git et GitHub ont été utilisés pour gérer les versions du projet, sauvegarder le code source et faciliter le suivi des modifications.

```
git add .
git commit -m "update␣tumorsight␣project"
git push origin main
```

Docker a été intégré afin de simplifier le déploiement et garantir la cohérence de l'environnement d'exécution.

Docker permet d'exécuter l'application dans un conteneur isolé, indépendamment du système d'exploitation utilisé.

```
docker build -t tumorsight .
docker run -p 8501:8501 tumorsight
```

8.1 Avantages du Déploiement avec Docker

- Portabilité de l'application.
- Reproductibilité de l'environnement.
- Facilité de déploiement.
- Isolation des dépendances.

Conclusion Générale

TumorSight démontre l'importance de l'intelligence artificielle dans le domaine de la santé et de l'imagerie médicale. Le projet combine des modèles de Deep Learning avec un tableau de bord interactif afin d'automatiser la détection des tumeurs cérébrales et la classification de leurs stades à partir d'images IRM.

La plateforme développée offre plusieurs fonctionnalités telles que le prétraitement des images, la prédiction automatique, la visualisation des scores de confiance, l'analyse par lot et l'exportation des résultats.

L'intégration des modèles CNN et EfficientNet permet d'étudier différentes approches de classification d'images médicales. Le CNN Baseline sert de modèle de référence, tandis qu'EfficientNet apporte une approche plus avancée grâce au Transfer Learning.

Même si le projet a obtenu des résultats prometteurs, plusieurs améliorations restent possibles, notamment l'utilisation de datasets plus grands, l'intégration de Grad-CAM, le support du format DICOM, le déploiement cloud et l'intégration avec les systèmes hospitaliers.

Globalement, TumorSight représente une solution intelligente et innovante montrant comment le Deep Learning peut contribuer à l'aide au diagnostic médical.

Références

— Documentation TensorFlow — https://www.tensorflow.org/

— Documentation Keras — https://keras.io/

— Documentation Streamlit — https://streamlit.io/

— Documentation Python — https://www.python.org/

— Documentation EfficientNet — https://keras.io/api/applications/efficientnet/